

D209: Data Mining

Task 2: Predictive Analysis

Hannah R. Bergman

WGU Email: hbergm3@wgu.edu

Table of Contents

Part I: Research Question.....	2
A1. Research Question.....	2
A2. Goal of the Data Analysis.....	2
Part II: Method Justification.....	3
B1. Predictive Method Technique & Expected Outcomes.....	3
How A Decision Tree Analyzes the Dataset for Diabetes Prediction.....	3
Expected Outcomes of the Analysis.....	3
B2. Assumption of the Chosen Prediction Method.....	3
B3. Python Packages & Libraries.....	3
Part III: Data Preparation.....	5
C1. Data Cleaning Goal.....	5
C2. Variable Selection.....	5
C3. Data Preparation Steps.....	6
Step 1. Detect & Handle Missing Values.....	6
Step 2. Outlier Detection & Handling.....	7
Step 3. Check & Fix Data Types.....	13
Step 4. Subset the Dataset.....	15
Step 5. Update Column Names.....	15
Step 6. Insert Dummies.....	16
C4. Cleaned Dataset.....	16
Part IV: Data Analysis.....	17
D1. Training & Testing Data Sets.....	17
D2. Analysis Technique.....	17
Step 1. Train/Test Split with Stratification.....	17
Step 2. Train a Base Decision Tree Using GridSearchCV.....	18
Step 3. Evaluate Decision Tree Performance.....	18
Step 4. Boost the Decision Tree with AdaBoost.....	20
Step 5. Evaluate AdaBoost Predictions.....	20

D3. Predictive Analysis Code.....	22
Part V: Data Summary & Implications.....	22
E1. Predictive Model Accuracy & MSE.....	22
Accuracy.....	22
Mean Squared Error (MSE).....	22
E2. Results & Implications of the Predictive Analysis.....	23
Results.....	23
Implications.....	23
E3. Limitation of the Analysis.....	23
E4. Recommended Course of Action.....	23
Part VI: Demonstration.....	24
F. Panopto Video Recording.....	24
G. Data & Code Sources.....	24
H. References.....	24

Part I: Research Question

A1. Research Question

Can a decision tree model be effectively employed to identify patients with diabetes?

Building upon my previous work in D208 Task 2 (Logistic Regression) and D209 Task 1 (K-Nearest Neighbors), this research aims to further refine diabetes prediction. By leveraging consistent data preparation techniques from those projects, I will explore the effectiveness of a **decision tree model** to enhance predictive accuracy.

A2. Goal of the Data Analysis

The primary goal of this analysis is to develop an effective predictive model for diabetes that ideally surpasses the performance of my previous model, which achieved 64% accuracy and an AUC of 0.5127. Accurate diabetes prediction is important for hospitals like ours, as it allows for proactive intervention to improve overall patient outcomes. Early identification allows for better management of existing health issues, reduces the risk of future complications, and ultimately improves patient outcomes. This proactive approach can lead to more efficient resource allocation, potentially shortening hospital stays, reducing readmissions, and preventing future hospitalizations due to diabetes-related complications.

Part II: Method Justification

B1. Predictive Method Technique & Expected Outcomes

How A Decision Tree Analyzes the Dataset for Diabetes Prediction

A decision tree works to analyze the dataset by building a hierarchical structure of questions, each with a basic binary (Yes/No) answer. Starting at the root node, the tree evaluates a specific feature and directs data points down corresponding branches. This branching continues, with each internal node posing a question based on a different feature, until a terminal node, or 'leaf,' is reached. The leaf node assigns a final classification, such as 'diabetes: Yes' or 'diabetes: No.'

Unlike linear models, a decision tree can capture more complex, non-linear relationships. This makes them particularly useful for datasets like ours, where diabetes risk factors interact in intricate ways. Compared to K-Nearest Neighbors (KNN), a decision tree requires no feature scaling, is resilient to outliers, and handles imbalanced data more effectively (Afeworki, 2021).

The tree's construction is driven by a process of feature selection and recursive splitting. At each node, the algorithm identifies the feature that best separates the data into groups with similar diabetes outcomes. For example, the initial split might be based on a key predictor like complication risk. Subsequent splits could refine these groups by considering age, BMI, and comorbidities like stroke or back pain. This iterative splitting continues until the tree accurately classifies the data, culminating in the leaf nodes that provide the final predictions.

Expected Outcomes of the Analysis

Our analysis will evaluate the model's performance using metrics such as accuracy, MSE, ROC and a confusion matrix. High performance across these metrics will indicate that our model effectively distinguishes between patients with and without diabetes.

B2. Assumption of the Chosen Prediction Method

Decision tree algorithms operate on the assumption that complex decisions can be broken down into a series of simpler, binary choices. At each node, decision trees employ binary splitting, dividing data into two groups based on a single feature - essentially asking a simple yes-or-no question (Anshul, 2025).

B3. Python Packages & Libraries

For this analysis, I'll be using Python - a programming language well suited for data science tasks like this. Python offers an extensive collection of powerful libraries and packages specifically designed to make working with data easier, including:

- **pandas (pd):** Offers a high-performance, user-friendly data structure (DataFrame) that mimics spreadsheets for easy, efficient data analysis and manipulation.

- **CategoricalDtype:** Enables the creation of pandas columns that accurately reflect the categorical or ordinal nature of our data.
- **NumPy (np):** Allows us to easily perform more complex mathematical computations within our dataset.
- **Matplotlib (plt):** Enables the creation of various plot types, such as line plots, scatter plots, histograms, and bar charts, for effective data visualization and model assessment.
- **Seaborn (sns):** Builds upon Matplotlib to help us produce more statistically-informed and aesthetically pleasing visualizations.
- **SciPy (scipy.stats)** Used for performing probabilistic distributions and statistical operations.
- **Scikit-learn (sklearn):** Provides access to essential tools for the machine learning aspect of our analysis, including:
 - **preprocessing:** Allows us to further preprocess our data, such as encoding and feature selection.
 - **train_test_split:** Splits our dataset into training and testing sets for the model evaluation part of our analysis.
 - **DecisionTreeClassifier:** Implements the decision tree algorithm for our analysis.
 - **GridSearchCV:** Performs hyperparameter tuning of the classifier.
 - **confusion_matrix:** Creates a confusion matrix to evaluate our model's performance.
 - **roc_auc_score:** Calculates the Area Under the ROC Curve (AUC) as a performance metric for our model.
 - **roc_curve:** Plots the Receiver Operating Characteristic (ROC) curve of our model.
 - **accuracy_score:** Calculates accuracy as a performance metric for our model.
 - **mean_squared_error:** Calculates the Mean Squared Error (MSE) as a performance metric for our model.
 - **classification_report:** Provides a concise summary of metrics to evaluate the overall performance of our model.

```
# Import the python packages & libraries
import pandas as pd
from pandas.api.types import import CategoricalDtype
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
sns.set()
from scipy import stats
from sklearn import preprocessing
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import confusion_matrix, roc_auc_score, roc_curve, accuracy_score, mean_squared_error, classifi
```

Part III: Data Preparation

C1. Data Cleaning Goal

One of our primary data preprocessing goals will be to convert our categorical variables into suitable numerical representations. While some tree-based algorithms can process categorical data directly, Scikit-Learn's `DecisionTreeClassifier`, our chosen model, requires numerical features. To account for this, we'll use one-hot encoding. This method creates binary columns for each category within a categorical variable. For example, the 'Complication_risk' feature, with the values of 'Low', 'Medium', and 'High', will be expanded into three binary columns: 'Complication_risk_Low', 'Complication_risk_Medium', and 'Complication_risk_High'. A value of 1 represents the presence of the corresponding category, and 0 indicates its absence.

C2. Variable Selection

- **Continuous (Numeric):**
 - Initial_days (days)
 - Income (USD)
 - Age (years)
 - TotalCharge (USD)
 - Children (#)
 - VitD_levels (#)
- **Categorical:**
 - Diabetes (Yes/No)
 - Gender (Male/Female/Nonbinary)
 - Complication_risk (High/Medium/Low)
 - Initial_admin (Emergency/Elective/Observation)

- Area (Urban/Suburban/Rural)
- Overweight (Yes/No)
- HighBlood (Yes/No)
- BackPain (Yes/No)
- Asthma (Yes/No)
- Stroke (Yes/No)

Our analysis focuses on a carefully selected subset of variables that directly align with our research question. These variables are anticipated to provide valuable insights into the factors influencing diabetes. While our dataset includes a broader array of variables, we will exclude some that are less relevant. For instance, demographic information such as city, state, county, zip code, latitude, longitude, population and area are purposefully excluded due to their irrelevance to our research.

C3. Data Preparation Steps

Step 1. Detect & Handle Missing Values

Check for and address any missing values in the dataset for our variables of interest.

Identification

- Utilize the `isnull()` function to check for missing values in our dataset.
- Employ the `sum()` method to return the total number of missing values in each column.

```
# Check for missing values in our dataset
df.isnull().sum()
```

```
CaseOrder      0
Customer_id    0
Interaction     0
UID            0
City           0
State          0
County         0
Zip            0
Lat            0
Lng            0
Population     0
Area           0
TimeZone       0
Job            0
Children       0
Age           0
Income         0
Marital        0
Gender         0
ReAdmis        0
VitD_levels    0
Doc_visits     0
Full_meals_eaten 0
vitD_supp      0
Soft_drink     0
Initial_admin  0
HighBlood      0
Stroke         0
Complication_risk 0
Overweight     0
Arthritis      0
Diabetes       0
Hyperlipidemia 0
BackPain       0
Anxiety        0
Allergic_rhinitis 0
Reflux_esophagitis 0
Asthma         0
Services       0
Initial_days   0
TotalCharge    0
Additional_charges 0
Item1          0
Item2          0
Item3          0
Item4          0
Item5          0
Item6          0
Item7          0
Item8          0
dtype: int64
```

Investigation

As we can see from the result above, there appears to be no missing values in our dataset.

Handling

The dataset does not have missing values for our variables of interest.

Step 2. Outlier Detection & Handling

Identify and address data points that significantly deviate from the expected pattern or range in our dataset. These outliers can skew our analysis results and lead to inaccurate conclusions.

Identification

- Utilize the `value_counts()` and `describe()` methods to identify any potential outliers.
- Focus on detecting data points that may indicate data entry errors or those that deviate significantly from the expected distribution.

```
# Frequency counts for categorical variable: Diabetes
df.Diabetes.value_counts()
No    7262
Yes    2738
Name: Diabetes, dtype: int64

# Frequency counts for categorical variable: Gender
df.Gender.value_counts()
Female    5018
Male      4768
Nonbinary    214
Name: Gender, dtype: int64

# Frequency counts for categorical variable: Complication_risk (complication risk level)
df.Complication_risk.value_counts()
Medium    4517
High      3358
Low        2125
Name: Complication_risk, dtype: int64

# Frequency counts for categorical variable: Initial_admin (admission type)
df.Initial_admin.value_counts()
Emergency Admission    5060
Elective Admission     2584
Observation Admission  2436
Name: Initial_admin, dtype: int64

# Frequency counts for categorical variable: Overweight
df.Overweight.value_counts()
Yes    7094
No     2966
Name: Overweight, dtype: int64

# Frequency counts for categorical variable: BackPain
df.BackPain.value_counts()
No    5886
Yes   4114
Name: BackPain, dtype: int64

# Frequency counts for categorical variable: HighBlood (high blood pressure)
df.HighBlood.value_counts()
No    5910
Yes   4090
Name: HighBlood, dtype: int64

# Frequency counts for categorical variable: Asthma
df.Asthma.value_counts()
No    7107
Yes   2093
Name: Asthma, dtype: int64

# Frequency counts for categorical variable: Stroke
df.Stroke.value_counts()
No    8807
Yes   1993
Name: Stroke, dtype: int64

# Frequency counts for categorical variable: Arthritis
df.Arthritis.value_counts()
No    6426
Yes   3574
Name: Arthritis, dtype: int64

# Frequency counts for categorical variable: Hyperlipidemia
df.Hyperlipidemia.value_counts()
No    6628
Yes   3372
Name: Hyperlipidemia, dtype: int64

# Frequency counts for categorical variable: Anxiety
df.Anxiety.value_counts()
No    6785
Yes   3215
Name: Anxiety, dtype: int64

# Frequency counts for categorical variable: Allergic_rhinitis
df.Allergic_rhinitis.value_counts()
No    6059
Yes   3941
Name: Allergic_rhinitis, dtype: int64

# Frequency counts for categorical variable: Area
df.Area.value_counts()
Rural    3369
Suburban 3328
Urban    3303
Name: Area, dtype: int64

# Frequency counts for categorical variable: Reflux_esophagitis
df.Reflux_esophagitis.value_counts()
No    5865
Yes   4135
Name: Reflux_esophagitis, dtype: int64

# Frequency counts for categorical variable: Soft_drink
df.Soft_drink.value_counts()
No    7425
Yes   2575
Name: Soft_drink, dtype: int64
```

```
# Summary statistics for continuous independent variables: Income, Age, TotalCharge (daily charge), Children (childr
continuous_summary = df[['Initial_days', 'Age', 'TotalCharge', 'Children', 'Income', 'VitD_levels', 'Doc_visits', 'F
continuous_summary
```

	Initial_days	Age	TotalCharge	Children	Income	VitD_levels	Doc_visits	Full_meals_eaten	vitD_supp
count	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	34.455299	53.511700	5312.172769	2.097200	40490.495160	17.964262	5.012200	1.001400	0.398900
std	26.309341	20.638538	2180.393838	2.163659	28521.153293	2.017231	1.045734	1.008117	0.628505
min	1.001981	18.000000	1938.312067	0.000000	154.080000	9.806483	1.000000	0.000000	0.000000
25%	7.896215	36.000000	3179.374015	0.000000	19598.775000	16.626439	4.000000	0.000000	0.000000
50%	35.836244	53.000000	5213.952000	1.000000	33768.420000	17.951122	5.000000	1.000000	0.000000
75%	61.161020	71.000000	7459.699750	3.000000	54296.402500	19.347963	6.000000	2.000000	1.000000
max	71.981490	89.000000	9180.728000	10.000000	207249.100000	26.394449	9.000000	7.000000	5.000000

Investigation

Determine the cause of the outlier - whether it's due to errors, anomalies, or legitimate extreme values.

- Diabetes:
 - While diabetes is a prevalent condition in this patient population, most (72.62%) of our patients do not have diabetes.
- Gender:
 - The patient population is predominantly female, with a smaller number of male patients. Nonbinary individuals represent a minority of the sample.
 - These findings indicate a gender imbalance in the dataset.

- **Complication_risk:**
 - The majority of patients (4517) were classified as Medium risk, followed by High risk (3358) and Low risk (2125).
 - These findings suggest a relatively balanced distribution of risk levels among the patient population.
- **Initial_admin:**
 - Emergency Admissions account for the majority (50.6%) of admissions, followed by Elective Admissions (25.0%) and Observation Admissions (24.4%).
 - This suggests that a majority of our patients required immediate medical attention.
- **Overweight:**
 - A significant majority (70.94%) of our patients are classified as overweight.
- **BackPain:**
 - Over half (58.86%) of the patients reported not having back pain, while 41.14% indicated experiencing back pain.
- **HighBlood:**
 - Over half (59.10%) of the patients do not have high blood pressure, while 40.90% do.
- **Asthma:**
 - While asthma is a prevalent condition in this patient population, most (71.07%) of our patients do not have asthma.
- **Stroke:**
 - A significantly higher proportion of patients (80.07%) have no reported history of stroke, while 19.93% do.
- **Arthritis:**
 - A higher proportion of patients (64.26%) have no reported history of Arthritis.
- **Hyperlipidemia:**

- A higher proportion of patients (66.28%) have no reported history of Hyperlipidemia.
- Anxiety:
 - A higher proportion of patients (67.85%) have no reported history of Anxiety.
- Allergic_rhinitis:
 - A higher proportion of patients (60.59%) have no reported history of Allergic Rhinitis.
- Area:
 - The amount of patients among various area types is relatively similar with 3,369 living in rural environments, 3,328 living in Suburban environments, and 3,303 living in Urban environments.
- Reflux_esophagitis:
 - About half of patients (58.65%) have no reported history of Reflux Esophagitis.
- Soft_drink:
 - A significant majority of patients (74.25%) have do not drink soft drinks.
- Initial_days
 - Average length of stay = ~34.5 days (with a standard deviation of ~26.3 days)
 - Minimum length of stay = ~1 day
 - Maximum length of stay = ~72 days
- VitD_levels
 - Average vit d level = ~17.96
- Doc_visits
 - Average number of doctors visits = ~5 visits
- Full_meals_eaten

- Average number of meals = ~ 1
- Age:
 - The absence of patients younger than 18 indicates that our facility either does not provide care to minors or that their data is not included in the current dataset. The maximum age of 89 aligns with typical human life expectancy.
- TotalCharge:
 - The typical average daily patient charge, without extra fees, is around \$5,312. Most charges are clustered near this average, with a standard deviation of about \$2,180. While the lowest and highest charges are both within two standard deviations from the average, suggesting a fairly even distribution, there's a slight imbalance toward one side. This is probably because of a few very high charges.
- Children:
 - While the average number of children per household is two, there is some noticeable variation - with some patients having no children in their household and others having larger households of up to ten children.
- Income:
 - There's a significant difference between the lowest income of \$154.08 and the highest reported income of \$207,249.10. The wide disparity between the minimum and maximum incomes could be a sign of outliers, such as very high or very low earners.

```
# Check for outliers in our Income data by looking at the largest and smallest values
income_largest = df['Income'].nlargest(30)
income_smallest = df['Income'].nsmallest(30)

print("LOWEST PATIENT INCOMES (Low to High):\n", income_smallest.to_string(index=False))
print("\nHIGHEST PATIENT INCOMES (High to Low):\n", income_largest.to_string(index=False))
```

LOWEST PATIENT INCOMES (Low to High):

```
154.08
300.79
395.23
401.86
493.04
695.22
702.16
798.98
826.01
881.07
881.40
953.74
1048.43
1078.12
1196.72
1209.56
1277.08
1286.25
1301.34
1366.98
1380.70
1392.23
1402.22
1409.92
1453.90
1480.67
1482.85
1486.44
1528.80
1541.63
```

HIGHEST PATIENT INCOMES (High to Low):

```
207249.10
204542.41
203774.60
197675.00
197576.18
196915.60
194796.24
190110.80
189416.27
189129.92
186791.40
186752.00
183037.10
179543.70
178945.40
178470.63
174745.85
171680.20
171288.05
171031.20
170961.50
167286.62
167105.10
166702.70
166048.50
166017.39
165588.50
165560.97
165329.55
164123.69
```

The income data shows a substantial range, encompassing individuals from very low-income to quite high-income backgrounds. While the majority of individuals fall within a typical income range, a small subset of individuals with exceptionally high

incomes may be identified as outliers. However, considering the potential for significant income growth and the absence of other relevant factors like geographical location, these extreme values are likely legitimate and not indicative of data errors.

Handling

Given our requirement for continuous quantitative variables and the ethical implications of using income to influence patient care, we'll retain the original income values in the dataset. Although this may introduce analytical challenges, it aligns with the project's objectives and avoids potential biases in healthcare outcomes.

Step 3. Check & Fix Data Types

Verify the data type of each column to ensure data consistency and compatibility with KNN classification.

Identification

- Determine the data type of each column using the dtype function.

```
# Check the data type of each column
df.dtypes
```

```
CaseOrder      int64
Customer_id    object
Interaction     object
UID            object
City           object
State          object
County         object
Zip            int64
Lat            float64
Lng            float64
Population     int64
Area           object
TimeZone       object
Job            object
Children       int64
Age            int64
Income         float64
Marital        object
Gender         object
ReAdmis        object
VitD_levels    float64
Doc_visits     int64
Full_meals_eaten int64
vitD_supp      int64
Soft_drink     object
Initial_admin  object
HighBlood      object
Stroke         object
Complication_risk object
Overweight     object
Arthritis      object
Diabetes        object
Hyperlipidemia object
BackPain       object
Anxiety        object
Allergic_rhinitis object
Reflux_esophagitis object
Asthma         object
Services       object
Initial_days   float64
TotalCharge    float64
Additional_charges float64
Item1          int64
Item2          int64
Item3          int64
Item4          int64
Item5          int64
Item6          int64
Item7          int64
Item8          int64
dtype: object
```

Verification

- Check if the data types align with the expected values and the requirements of the algorithm:

- Numeric variables: should generally be of type int or float
- Categorical variables: May need to be converted to a suitable numeric representation

The data types of this dataset appear mostly suitable for our analysis.

- **Continuous Variables:** Initial_days (float), Income (float), Age (int), and Children (int) are appropriately defined as numerical data types, already, allowing for quantitative analysis.
- **Categorical Variables:** Overweight, Diabetes, HighBlood, BackPain, Asthma, Stroke, Gender, Complication_risk, and Initial_admin are in a format suitable for encoding into categorical representations.

Handling

Scikit-Learn's DecisionTreeClassifier operates on numerical data. To accommodate the presence of categorical and boolean variables in our dataset, several preprocessing steps are necessary, including:

Numeric Adjustment:

- Convert Income data from decimal format to whole numbers by removing fractional parts.

```
# Cast Income column data type from float to integer
df["Income"] = df["Income"].astype(int)
```

- Round down TotalCharge & VitD_levels to two decimal places to better reflect dollar amounts and for improved readability.

```
# Reduce column precision down to 2 decimal places to better reflect USD pricing and make vitamin d level numbers mo
df["TotalCharge"] = df.TotalCharge.round(2)
df["VitD_levels"] = df.VitD_levels.round(2)
```

Categorical Variable Conversion:

- Convert Gender, Complication_risk, Initial_admin, and Area to categorical data types to accurately represent their respective values.

```
# Cast column data types from string to category
df["Gender"] = df["Gender"].astype("category")
df["Complication_risk"] = df["Complication_risk"].astype("category")
df["Initial_admin"] = df["Initial_admin"].astype("category")
df["Area"] = df["Area"].astype("category")
```

- Use one-hot encoding with the get_dummies() function to create binary columns for each category, ensuring compatibility with the DecisionTreeClassifier.
- To prevent multicollinearity, drop the first category in each categorical variable using drop_first=True. This category will serve as a baseline.

```
# Create dummy variables, dropping the first column to avoid multicollinearity
df_gender = pd.get_dummies(data=df["Gender"], drop_first=True)
df_admission_type = pd.get_dummies(data=df["Initial_admin"], drop_first=True)
df_complication_risk_level = pd.get_dummies(data=df["Complication_risk"], drop_first=True)
df_area_type = pd.get_dummies(data=df["Area"], drop_first=True)
```

Boolean Variable Conversion:

- Convert boolean values (1 for "Yes" and 0 for "No") using a conversion dictionary. This will ensure compatibility with the numerical requirements of the DecisionTreeClassifier.

```
# Define the boolean conversion dictionary
boolean_conversion = {"Yes" : 1, "No" : 0}

# Map string objects to respective boolean values, changing data type to int
df["Overweight"] = df["Overweight"].map(boolean_conversion)
df["Diabetes"] = df["Diabetes"].map(boolean_conversion)
df["HighBlood"] = df["HighBlood"].map(boolean_conversion)
df["BackPain"] = df["BackPain"].map(boolean_conversion)
df["Asthma"] = df["Asthma"].map(boolean_conversion)
df["Arthritis"] = df["Arthritis"].map(boolean_conversion)
df["Stroke"] = df["Stroke"].map(boolean_conversion)
df["Hyperlipidemia"] = df["Hyperlipidemia"].map(boolean_conversion)
df["Anxiety"] = df["Anxiety"].map(boolean_conversion)
df["Allergic_rhinitis"] = df["Allergic_rhinitis"].map(boolean_conversion)
df["Reflux_esophagitis"] = df["Reflux_esophagitis"].map(boolean_conversion)
df["Soft_drink"] = df["Soft_drink"].map(boolean_conversion)
```

Step 4. Subset the Dataset

To focus on the relevant variables for our model, we'll create a new dataset called `df_subset` that includes only the columns we plan to use as features. This will help streamline our analysis and improve computational efficiency.

```
# Subset the dataset, selecting only the columns relevant to our research question
relevant_columns = [
    "Initial_days",
    "Age",
    "Income",
    "Children",
    "Overweight",
    "Diabetes",
    "BackPain",
    "Asthma",
    "HighBlood",
    "Stroke",
    "Arthritis",
    "Hyperlipidemia",
    "Anxiety",
    "Allergic_rhinitis",
    "Reflux_esophagitis",
    "Soft_drink",
    "TotalCharge",
    "VitD_levels",
    "Doc_visits",
    "Full_meals_eaten",
    "vitD_supp"
]

df_subset = df[relevant_columns]
```

Step 5. Update Column Names

Rename column labels to provide more descriptive and meaningful names that accurately reflect the data they contain. This will enhance data clarity and prevent

misunderstandings when working with the dataset. Ensure that the new labels are consistent in style and format.

```
# Update column labels for better understanding and consistency
updated_columns = [
    "length_of_stay",
    "age",
    "income",
    "children_in_household",
    "overweight",
    "diabetes",
    "back_pain",
    "asthma",
    "high_bp",
    "stroke",
    "arthritis",
    "hyperlipidemia",
    "anxiety",
    "allergic_rhinitis",
    "reflux_esophagitis",
    "soft_drink_consumer",
    "daily_charge",
    "vitamin_d_level",
    "doctor_visits",
    "full_meals_eaten",
    "vitamin_d_supplements"
]

df_subset = df_subset.set_axis(updated_columns, axis=1)
```

Step 6. Insert Dummies

After constructing our new dataframe, we can use the insert function to strategically place our newly generated dummy variables within the desired columns, ensuring they align with the existing data structure and facilitate subsequent analysis.

```
# Insert dummy variables for the 'complication_risk_level' categorical variable
df_subset.insert(4, "complication_risk_level_medium", df_complication_risk_level["Medium"])
df_subset.insert(4, "complication_risk_level_low", df_complication_risk_level["Low"])

# Insert dummy variables for the 'admission_type' categorical variable
df_subset.insert(4, "admission_type_emergency", df_admission_type["Emergency Admission"])
df_subset.insert(4, "admission_type_observation", df_admission_type["Observation Admission"])

# Insert dummy variables for the 'gender' categorical variable
df_subset.insert(3, "gender_nonbinary", df_gender["Nonbinary"])
df_subset.insert(3, "gender_male", df_gender["Male"])

# Insert dummy variables for the 'area_type' categorical variable
df_subset.insert(3, "area_type_urban", df_area_type["Urban"])
df_subset.insert(3, "area_type_suburban", df_area_type["Suburban"])
```

C4. Cleaned Dataset

The prepared dataset, **'task2_transformed_medical_clean.csv'**, can be found within the main project directory and is also included as an attachment to this report.

```
# Save the transformed dataset as a CSV file
df_subset.to_csv('task2_transformed_medical_clean.csv', index=False)

print(f"The prepared dataset has been saved as:\n ► 'task2_transformed_medical_clean.csv'")
```

The prepared dataset has been saved as:
► 'task2_transformed_medical_clean.csv'

Part IV: Data Analysis

D1. Training & Testing Data Sets

The training and testing data sets can be found within the main project directory and are also included as attachments to this report. The split datasets are saved as:

- **'X_train: task2_Xtrain.csv'**
- **'X_test: task2_Xtest.csv'**
- **'y_train: task2_ytrain.csv'**
- **'y_test: task2_ytest.csv'**

```
# Define X (features) and y (response variable)
X = df_subset.drop(["diabetes"], axis=1).copy()
y = df_subset["diabetes"].copy()

# Split the data into train and test sets (80% train, 20% test) with stratification
X_train, X_test, y_train, y_test = train_test_split(
    X, y, train_size=0.8, test_size=0.2, random_state=42, stratify=y
)
```

```
# Save datasets as CSV files
file_paths = {
    "X_train": "task2_Xtrain.csv",
    "X_test": "task2_Xtest.csv",
    "y_train": "task2_ytrain.csv",
    "y_test": "task2_ytest.csv",
}

X_train.to_csv(file_paths["X_train"], index=False)
X_test.to_csv(file_paths["X_test"], index=False)
y_train.to_csv(file_paths["y_train"], index=False)
y_test.to_csv(file_paths["y_test"], index=False)
```

```
print("The split datasets have been saved as:")
for name, path in file_paths.items():
    print(f"  ▶ {name}: {path}")
```

```
The split datasets have been saved as:
  ▶ X_train: task2_Xtrain.csv
  ▶ X_test: task2_Xtest.csv
  ▶ y_train: task2_ytrain.csv
  ▶ y_test: task2_ytest.csv
```

D2. Analysis Technique

Step 1. Train/Test Split with Stratification

Before training our model, we need to split our dataset into training and testing subsets (which we performed previously in Section D1) to accurately assess performance and avoid overfitting. Given the class imbalance in our target variable (diabetes), with approximately 27% positive cases, we utilized stratified sampling. This technique helps guarantee that the proportion of diabetes cases is consistent across

both training and testing sets, enabling a more reliable evaluation of the model's predictive capabilities.

Step 2. Train a Base Decision Tree Using GridSearchCV

Decision trees can overfit easily if left untuned, so we'll use GridSearchCV to help optimize parameters (tree depth, minimum samples per leaf) using cross-validation. This ensures our model is the best generalizable version, not just one that performs well on our training data. It helps prevent issues like overly complex trees (which memorize data) and oversimplified trees (which can miss meaningful patterns in our data). This provides us with a better balanced, well-tuned baseline model that will serve as our reference point, and also gives us some insight into how sensitive performance is to tree complexity.

```
# Decision Tree with GridSearchCV

# Define the base Decision Tree model
dtree = DecisionTreeClassifier(random_state=42)

# Define a parameter grid for tuning
param_grid = {
    'max_depth': [3, 5, 10, None],
    'min_samples_leaf': [1, 2, 4]
}

# Setup GridSearchCV
grid_search = GridSearchCV(
    estimator=dtree,
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

# Fit GridSearchCV on training data
grid_search.fit(X_train, y_train)

# Best Decision Tree model
best_tree = grid_search.best_estimator_
print("Best Decision Tree Parameters:", grid_search.best_params_)

Best Decision Tree Parameters: {'max_depth': 3, 'min_samples_leaf': 1}
```

Step 3. Evaluate Decision Tree Performance

A confusion matrix allows us to analyze the distribution of false positives and false negatives, highlighting potential areas for improvement. We supplement this with a classification report, which provides precision, recall, and F1-score, and calculate MSE/RMSE and AUC to assess model fit and class separation capabilities. This evaluation provides a good understanding of the baseline model's strengths and weaknesses.

```

# Evaluate Decision Tree Predictions

# Predictions and Probabilities from the tuned Decision Tree
y_pred_tree = best_tree.predict(X_test)
y_proba_tree = best_tree.predict_proba(X_test)[:, 1]

# Evaluation Metrics

# Classification Report
print("Classification Report (Decision Tree):\n", classification_report(y_test, y_pred_tree))

# Confusion Matrix
print("Confusion Matrix (Decision Tree):")
sns.heatmap(confusion_matrix(y_test, y_pred_tree), annot=True, fmt='d', cmap='Blues')
plt.title("Decision Tree Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# MSE and RMSE
mse_tree = mean_squared_error(y_test, y_pred_tree)
rmse_tree = np.sqrt(mse_tree)
print(f"Decision Tree MSE: {mse_tree:.4f}")
print(f"Decision Tree RMSE: {rmse_tree:.4f}")

# AUC Score
auc_tree = roc_auc_score(y_test, y_proba_tree)
print(f"Decision Tree AUC Score: {auc_tree:.4f}")

```

```

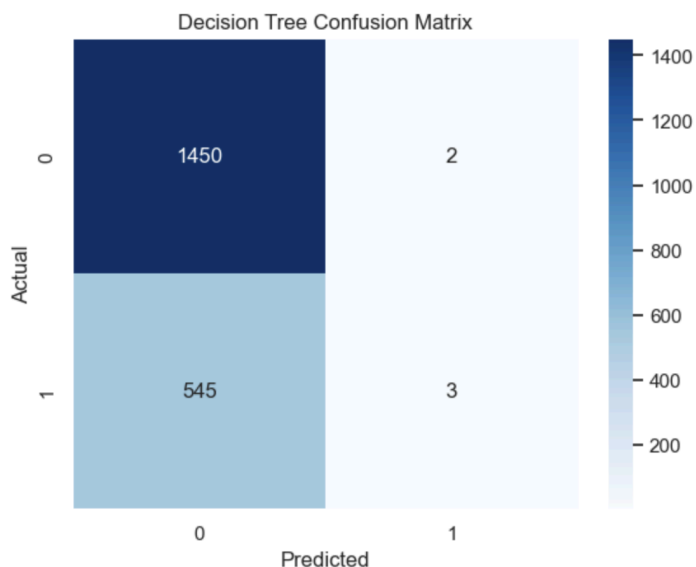
Classification Report (Decision Tree):
              precision    recall  f1-score   support

     0       0.73         1.00         0.84         1452
     1       0.60         0.01         0.01          548

 accuracy          0.66
 macro avg         0.66         0.50         0.43         2000
 weighted avg      0.69         0.73         0.61         2000

```

Confusion Matrix (Decision Tree):



```

Decision Tree MSE: 0.2735
Decision Tree RMSE: 0.5230
Decision Tree AUC Score: 0.5204

```

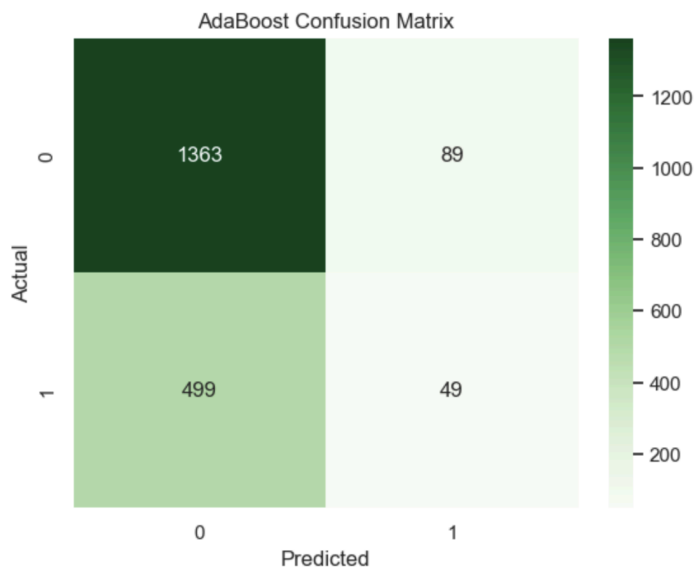
Through GridSearchCV, we determined the optimal decision tree configuration: max_depth=3 and min_samples_leaf=1. This configuration resulted in an AUC of

0.5204. Though this means our model exceeds random classification, it only does so by a marginal amount. This result suggests the decision tree is a weak learner, struggling to capture complex patterns.

Classification Report (AdaBoost):

	precision	recall	f1-score	support
0	0.73	0.94	0.82	1452
1	0.36	0.09	0.14	548
accuracy			0.71	2000
macro avg	0.54	0.51	0.48	2000
weighted avg	0.63	0.71	0.64	2000

Confusion Matrix (AdaBoost):



AdaBoost MSE: 0.2940
AdaBoost RMSE: 0.5422
AdaBoost AUC Score: 0.5778

D3. Predictive Analysis Code

Please refer to the attached file **'hrbergman_d209_task2.ipynb'** for the code used to perform the classification analysis.

Part V: Data Summary & Implications

E1. Predictive Model Accuracy & MSE

Accuracy

Our analysis revealed a baseline decision tree accuracy of 73% (0.73), compared to 71% (0.71) for the AdaBoost (boosted) model. While this represents a slight reduction,

it's consistent with AdaBoost's tendency to optimize for minority class performance (Peters, 2023). In our case, this suggests the model may be more sensitive to identifying true diabetic cases, potentially increasing false positives in the non-diabetic group. Given the dataset's class imbalance (fewer diabetic patients), it should be noted that accuracy alone is an unreliable indicator of its overall performance.

```
# Baseline decision tree classification report
print("Classification Report (Decision Tree):\n", classification_report(y_test, y_pred_tree))

# Boosted decision tree classification report
print("Classification Report (AdaBoost):\n", classification_report(y_test, y_pred_ada))
```

Classification Report (Decision Tree):				
	precision	recall	f1-score	support
0	0.73	1.00	0.84	1452
1	0.60	0.01	0.01	548
accuracy			0.73	2000
macro avg	0.66	0.50	0.43	2000
weighted avg	0.69	0.73	0.61	2000

Classification Report (AdaBoost):				
	precision	recall	f1-score	support
0	0.73	0.94	0.82	1452
1	0.36	0.09	0.14	548
accuracy			0.71	2000
macro avg	0.54	0.51	0.48	2000
weighted avg	0.63	0.71	0.64	2000

Mean Squared Error (MSE)

The baseline decision tree achieved an MSE of 0.2735, while the AdaBoost model's MSE was 0.2940. This represents a marginal increase in the average squared prediction error, roughly a ~2% difference. This slight increase in MSE for AdaBoost could be a consequence of its emphasis on correctly classifying minority class instances, which may result in larger errors for certain predictions.

```
# Baseline decision tree MSE
print(f"Decision Tree MSE: {mse_tree:.4f}")

# Boosted decision tree MSE
print(f"AdaBoost MSE: {mse_ada:.4f}")
```

Decision Tree MSE: 0.2735
AdaBoost MSE: 0.2940

E2. Results & Implications of the Predictive Analysis

Results

Circling back to my initial research question, I had originally intended to leverage AdaBoost to improve upon the Decision Tree's predictive performance in order to effectively identify patients with diabetes. However, while the Decision Tree achieved higher accuracy (73%) and lower MSE, it failed to adequately identify diabetic patients, achieving a recall of only 1%. This highlights a significant class imbalance issue, rendering the model clinically unacceptable. Conversely, AdaBoost, though slightly less accurate (71%) and exhibiting a higher MSE, dramatically improved recall to 9%, with

corresponding improvements in F1-score (0.14) and AUC (0.58). While this resulted in more false positives, this is a more acceptable outcome in healthcare, where the cost of missed diagnoses is high. Thus, despite lower overall accuracy, AdaBoost offers a more balanced and clinically useful model.

Implications

While our boosted model performs marginally better, it is still unsuitable for practical use. To further optimize for clinical, real-world application, future efforts should focus on making more improvements, such as enhancing sensitivity through resampling and threshold adjustments.

E3. Limitation of the Analysis

One major limitation of this analysis is the significant class imbalance in the target variable. This imbalance has strongly skewed overall model performance, resulting in high overall accuracy driven primarily by the accurate classification of non-diabetic cases. This came at the cost of poor sensitivity for diabetic patients, leading to unacceptably high false negative rates. As a result, the models are inadequate for practical applications such as early diabetes detection or screening.

E4. Recommended Course of Action

Given the disappointing performance of the boosted model, particularly its lower accuracy and higher propensity to error compared to the initial model, it needs much more refinement to ensure reliability in real-world application to effectively identify patients with diabetes.

Throughout this analysis and our previous projects, we've discovered our models tend to exhibit a significant bias towards classifying patients as non-diabetic, resulting in zero true and false positives. This bias is likely due to a class imbalance issue, stemming from the overrepresentation of non-diabetic patients in our dataset.

To mitigate this, we can implement resampling techniques such as oversampling (increasing the number of diabetic patients we have in our training data) or undersampling (reducing the number of non-diabetic patients in our training data). We can also adjust class weights to penalize misclassifications of diabetic cases, to help encourage our model to focus on detecting diabetic patients.

Additionally, we should consider evaluating our model using other metrics that are more suitable for imbalanced data, such as the Precision-Recall Curve and F1 Score (rather than Accuracy and MSE). These metrics focus on how well the model identifies diabetic cases rather than relying solely on accuracy, providing a more comprehensive assessment of performance.

Part VI: Demonstration

F. Panopto Video Recording

Panopto video recording:

<https://wgu.hosted.panopto.com/Panopto/Pages/Viewer.aspx?id=ebd7b7e1-c445-41ee-9876-b2a40133f88c>

G. Data & Code Sources

Western Governors University. (2023). *Medical Data and Dictionary Files* [Dataset]. Data Files and Associated Dictionary Files.

Retrieved January 11, 2025, from

<https://access.wgu.edu/ASP3/aap/content/q9rke9s0rlc9ejd92md0.html>

W3Schools. (2024). *Machine Learning- Confusion Matrix*. [Source code]. Creating a Confusion Matrix.

Retrieved January 11, 2025, from

[w3schools.com/python/python_ml_confusion_matrix.asp](https://www.w3schools.com/python/python_ml_confusion_matrix.asp)

H. References

Afeworki, M. (2021, September 9). *A Comparison of Machine Learning Algorithms: KNN vs. Decision Trees*. Medium.

Retrieved March 6, 2025, from

medium.com/a-comparison-of-machine-learning-algorithms

Anshul. (2025, February 17). *What is Decision Tree? | Decision Tree Assumptions*. Analytics Vidhya.

Retrieved March 6, 2025, from

analyticsvidhya.com/blog/2021/08/decision-tree-algorithm

Bhandari, A. (2024, September 18). *What is Multicollinearity? | Causes, Effects and Detection Using VIF*. Analytics Vidhya.

Retrieved February 1, 2024, from

analyticsvidhya.com/blog/2020/03/what-is-multicollinearity

Boorman, G., Peterson, A., Chapman, J., & Weber, I. (n.d.). *Supervised Learning with scikit-learn*. DataCamp.

Retrieved January 23, 2025 from

datacamp.com/learn/courses/supervised-learning-with-scikit-learn

CDC (2024, July 23). *National Diabetes Statistics Report*. Fast Facts on Diabetes.

Retrieved January 11, 2025, from

cdc.gov/diabetes/php/data-research/index.html

Hunter, J., Dale, D., Firing, E., Droettboom, M., & Matplotlib Development Team. *Choosing Colormaps in Matplotlib*. Matplotlib.

Retrieved February 1, 2024, from

matplotlib.org/stable/users/explain/colors/colormaps.html#id4

Jayaswal, V. (2020, September 14). *Performance Metrics: Confusion matrix, Precision, Recall, and F1 Score*. Towards Data Science.

Retrieved February 1, 2024 from

towardsdatascience.com/performance-metrics

John Hopkins Medicine. (2023, November 3). *Diabetes and High Blood Pressure*.

Retrieved February 1, 2024 from

hopkinsmedicine.org/health/conditions-and-diseases/diabetes

Peters, Mirko. (2023, October 18). *Boosting Performance with AdaBoost in Machine Learning*. Medium.

Retrieved March 11, 2025 from

mirkopeters.com/boosting-performance-with-adaboost-in-machine-learning

Ragan, A. (2018, October 10). *Taking the Confusion Out of Confusion Matrices*. Towards Data Science.

Retrieved February 1, from

towardsdatascience.com/taking-the-confusion-out-of-confusion-matrices